

Практическое задание №3

StatefulSet, Volumes, GatewayAPI

- Работу продолжаем делать на базе 2-ой работы
- Доработайте приложение, чтобы для его работы требовалось сохранять и отображать данные (любые) в базе данных. БД используйте PostgreSQL
- Разверните PostgreSQL в K8s, используя StatefulSet + Volume&VolumeMounts. Запрещено использовать готовые операторы типа CloudNativePG или Zalando, а так же готовые чарты
- Переведите приложение на GatewayAPI вместо Ingress и Ingress Controller

Введение

Цель работы: Данная лабораторная работа является итерационным развитием инфраструктурного стека, построенного в ходе предыдущих этапов. Если целью второй работы было освоение базовых механизмов оркестрации Stateless-приложений, то текущая задача направлена на проектирование **Stateful-систем** (систем с состоянием) и переход к современным стандартам сетевого взаимодействия в Kubernetes.

В ходе выполнения работы были выполнены следующие шаги:

Освоение Stateful-архитектуры: Реализация управления базами данных внутри Kubernetes с использованием StatefulSet. В отличие от Deployment, данный контроллер обеспечивает уникальную идентификацию экземпляров и строгую последовательность запуска, что критически важно для работы распределенных хранилищ данных.

Обеспечение сохранности данных: Настройка связки PersistentVolume (PV) и PersistentVolumeClaim (PVC) для того, чтобы данные приложения не терялись при перезапуске подов или обновлении узлов кластера.

Внедрение Gateway API: Изучение нового стандарта маршрутизации в Kubernetes. В отличие от Ingress, Gateway API разделяет ответственность между администраторами инфраструктуры (объекты GatewayClass, Gateway) и разработчиками приложений (HTTPRoute), что является лучшей практикой в Enterprise-средах.

Разработка системы с состоянием: Доработка исходного кода приложения на Python/Flask для реализации транзакционных операций: записи посещений пользователей в БД и отображения истории запросов.

Ход работы.

Приложение на Python было адаптировано для работы с базой данных.

- **ConfigMap:** В объект ConfigMap были вынесены параметры подключения: DB_HOST, DB_USER, DB_NAME. Это позволило не менять исходный код при деплое в разные окружения.
- **Deployment:** Приложение запущено как Deployment с переменными окружения, ссылающимися на значения из ConfigMap.
- **Инициализация:** При первом запуске приложение обращается к функции init_db(), которая проверяет наличие таблицы visits и создает её при необходимости.

Реализованы следующие .yaml файлы:

Configmap.yaml:

```
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: hello-config
5  data:
6    NAME: Nikita
```

Deployment.yaml:

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: hello-deployment
5  spec:
6    replicas: 2
7    selector:
8      matchLabels:
9        app: hello
10   template:
11     metadata:
12       labels:
13         app: hello
14   spec:
15     containers:
16     - name: hello
17       image: vegetarianiez/hello-k8s:v2
18       ports:
19         - containerPort: 5001
20       env:
21         - NAME = <valueFrom>
22         - DB_HOST = postgres-service
23         - DB_NAME = hello_db
24         - DB_USER = postgres
25         - DB_PASSWORD = <valueFrom>
```

Gateway.yaml:

```
1  apiVersion: gateway.networking.k8s.io/v1
2  kind: Gateway
3  metadata:
4    name: my-gateway
5  spec:
6    gatewayClassName: envoy
7    listeners:
8      - name: http
9        protocol: HTTP
10       port: 80
11       allowedRoutes:
12         namespaces:
13           from: Same
14   ---
15   apiVersion: gateway.networking.k8s.io/v1
16   kind: HTTPRoute
17   metadata:
18     name: hello-route
19   spec:
20     parentRefs:
21       - name: my-gateway
22     rules:
23       - matches:
24         - path:
25             type: PathPrefix
26             value: /
27         backendRefs:
28           - name: hello-service
29             port: 80
```

hello-service.yaml:

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: hello-service
5  spec:
6    selector:
7    app: hello
8  ports:
9    - protocol: TCP
10     port: 80
11     targetPort: 5001
```

Postgres.yaml:

```
19  apiVersion: apps/v1
20  kind: StatefulSet
21  metadata:
22    name: postgres
23  spec:
24    serviceName: postgres-service
25    replicas: 1
26    selector:
27    matchLabels:
28      app: postgres
29    template:
30      metadata:
31      labels:
32        app: postgres
33      spec:
34        containers:
35        - name: postgres
36          image: postgres:15
37          ports:
38            - containerPort: 5432
39          env:
40            - name: POSTGRES_PASSWORD
41              valueFrom:
42                secretKeyRef:
43                  name: postgres-secret
44                  key: POSTGRES_PASSWORD
45            - name: POSTGRES_DB
46              value: hello_db
47          volumeMounts:
48            - name: pgdata
49              mountPath: /var/lib/postgresql/data
```

Описание: Стандартные Ingress-манифесты из второй лабораторной работы перестали работать после установки Gateway API.

Причина: Gateway API требует предварительной установки CRD (Custom Resource Definitions) в кластер и наличия активного Gateway-контроллера (например, Cilium, Istio или Nginx Gateway API).

Решение:

1. Проверка установки API с помощью команды `kubectl api-resources | grep gateway`.
2. Реализация цепочки: GatewayClass (определяет драйвер) -> Gateway (создает Listener на порту 80) -> HTTPRoute (направляет трафик). Эта структура позволила отделить «сетевую политику» (Gateway) от «логики маршрутизации» (HTTPRoute).

Сравнительный анализ архитектур (Stateful vs Stateless):

Характеристика	Stateless	Stateful
Хранение данных	Нет (данные временные)	Да (диски/PVC)
Жизненный цикл	Легко заменить любым экземпляром	Требует сохранения идентификации (StatefulSet)
Пример сервиса	Web-фронтенд, API-сервис	БД, системы очередей (RabbitMQ)
Сложность развертывания	Низкая	Высокая (требует управления томами)

Основные добавления и изменения

Изменение 1: Интеграция слоя данных (Persistence Layer)

- Было: Приложение работало автономно, вся информация (если была) терялась при удалении пода.
- Стало: Добавлен слой хранения данных на базе PostgreSQL. Реализована связка:
 - PersistentVolumeClaim (PVC): Запрос на выделение места.
 - StatefulSet: Контроллер, обеспечивающий уникальность и порядок запуска экземпляров БД.
 - VolumeMounts: Физическая привязка каталога базы данных внутри контейнера к выделенному диску.

Изменение 2: Модернизация сетевой логики (Network Layer)

- Было: Использование Ingress — стандартного, но менее гибкого инструмента, управляющего трафиком через аннотации.

- Стало: Внедрение Gateway API. Это архитектурное изменение разделило сетевую инфраструктуру на независимые объекты:
 - Gateway: Управляет точкой входа (листенеры, порты, сертификаты).
 - HTTPRoute: Управляет логикой маршрутизации трафика к конкретным сервисам.
 - Результат: Такая структура позволяет администратору сети управлять доступом, не вмешиваясь в манифесты разработчика приложения.

Изменение 3: Логика взаимодействия микросервисов

- Было: Взаимодействие ограничено рамками одного приложения.
- Стало: Внедрена модель Service Discovery. Приложение Flask теперь динамически находит базу данных через DNS-имя postgres-service, что позволяет системе работать даже при смене IP-адресов подов внутри кластера.

Заключение.

В ходе выполнения второй лабораторной работы была успешно решена задача развертывания веб-приложения в кластере Kubernetes. Переход от инструментов локальной контейнеризации к системам промышленного уровня (оркестрации) позволил на практике изучить принципы управления распределенными системами и подтвердил эффективность декларативного подхода к администрированию инфраструктуры.

Полученный опыт работы с объектами Deployment, ConfigMap и Ingress стал фундаментом для понимания архитектуры современных высоконагруженных систем. Мы убедились, что декларативный подход не только повышает предсказуемость развертываний, но и значительно упрощает процесс масштабирования и обслуживания приложений в условиях реальной эксплуатации.

Созданный прототип является идеальной базой для дальнейшего внедрения практик CI/CD, настройки автомасштабирования (HPA) и мониторинга системы с использованием инструментов Prometheus и Grafana. Изученные принципы оркестрации позволяют уверенно переходить к проектированию более сложных систем, основанных на принципах Cloud Native.